AI Agent Governance and Observability in Practice

Control what the agent can do.

Understand what actually happened.

Xiaodi Yan · Microsoft MVP

Python · OpenTelemetry · one support agent

25 minutes

The answer is only part of the story

> “I couldn’t complete that request.”

Was the action blocked?

The policy may have worked exactly as intended.

Did the action fail?

The policy may have allowed it, but a dependency timed out.

The same final message can hide very different runs.

Two capabilities, one application

Governance

Is this action allowed?

- Policy and permissions
- Tool access and approval
- Host enforcement

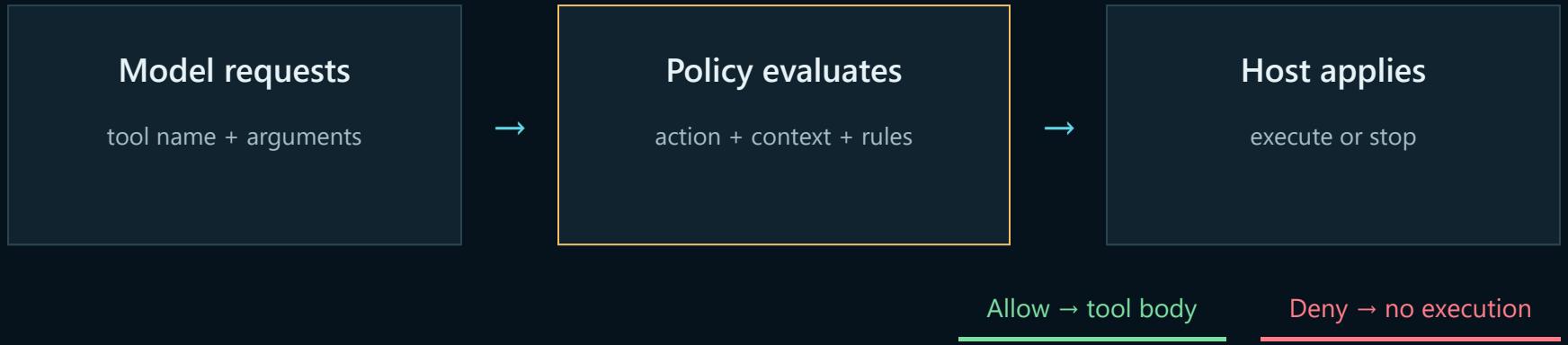
Observability

What actually happened?

- Execution traces
- Correlated logs
- Latency, tokens and outcomes

Identity, auditing and evaluation are part of the wider system. Today we focus on these two capabilities.

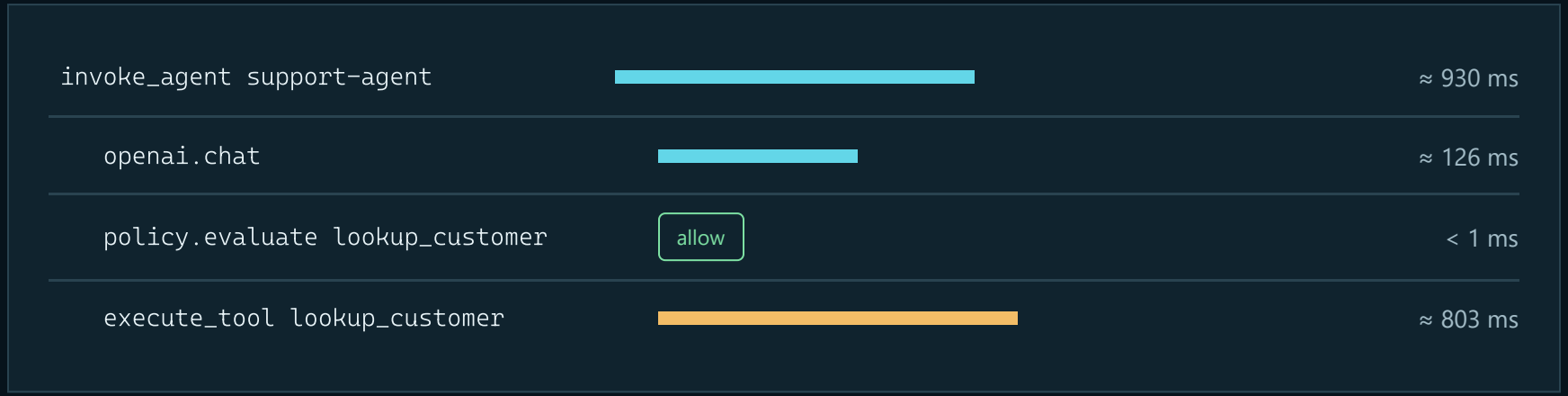
Control belongs at the action boundary



A model can request an action. The host controls execution.

This demo uses the official Agent Governance Toolkit policy engine. The host enforces its decision.

Read a run, then ask better questions



Traces

Where did this run spend time?

Logs

Which rule or error explains it?

Metrics

How often does this happen?

Representative offline rehearsal timings; values vary. These are operations and outcomes, not a model's private reasoning.

One trace contains related spans.

A common language for AI telemetry

GenAI semantic conventions

`gen_ai.operation.name`	chat
`gen_ai.request.model`	demo-model
`gen_ai.usage.input_tokens`	80
`gen_ai.usage.output_tokens`	18

Shared names make data easier to compare.

Conventions
Define the meaning and structure of signals.

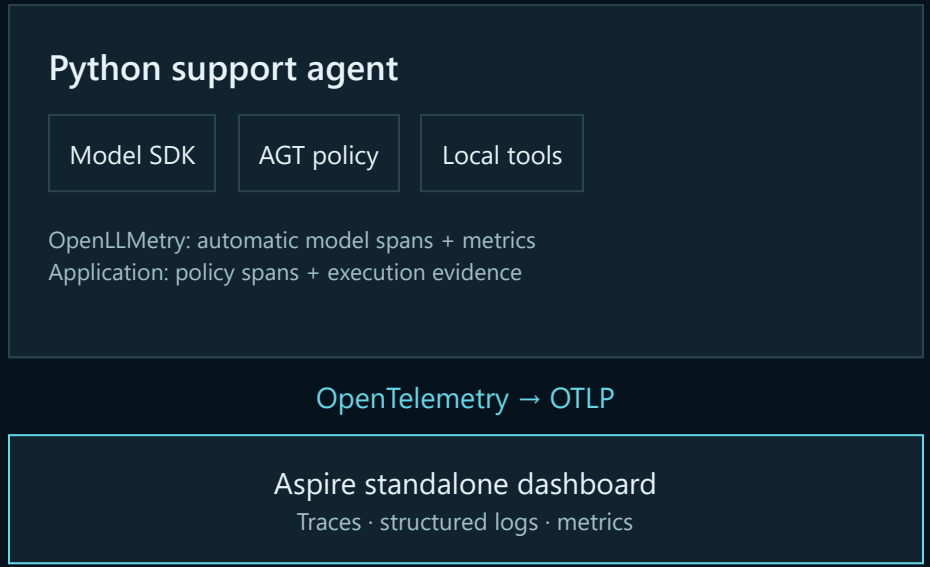
OpenLLMetry
Instruments model SDK calls.

OpenTelemetry
SDKs, context and export pipelines.

Dashboard
Queries and visualizes the data.

GenAI conventions are in Development. Pin your libraries and verify their actual output; names evolve.

Instrument the model. Record the boundary.




```python
# Explicitly disable content capture in this demo.
os.environ["TRACELOOP_TRACE_CONTENT"] = "false"
OpenAIInstrumentor().instrument(
    tracer_provider=traces,
    meter_provider=meters,
)
```

Use the individual OpenLLMetry instrumentation with your own OTel pipeline.
No Traceloop account is required.

One instrumentation owner for each model call prevents duplicate spans and token counts.

SDK-level demonstration; no framework-specific dependency.

Live demo · 1

Open the support agent

Allowed: the tool executes

> “Look up customer C-1042 and summarize their support status.”

```
invoke_agent
    openai.chat
    policy.evaluate  allow
    execute_tool lookup_customer
    openai.chat → answer
```

Check the evidence

Model and tool timings

Input / output token usage

Policy log with the same trace ID

1

Lookup tool body execution

Model request → policy allow → execution → answer

Live demo · 2

Request denied deletion

Blocked: the control worked

> “Delete customer record C-1042.”

```
invoke_agent
    openai.chat
    policy.evaluate   deny
```

Execution stops before the tool body.

Check the evidence

`block-destructive`

Policy outcome: **deny**

Agent outcome: **blocked**

0

Deletion tool body executions

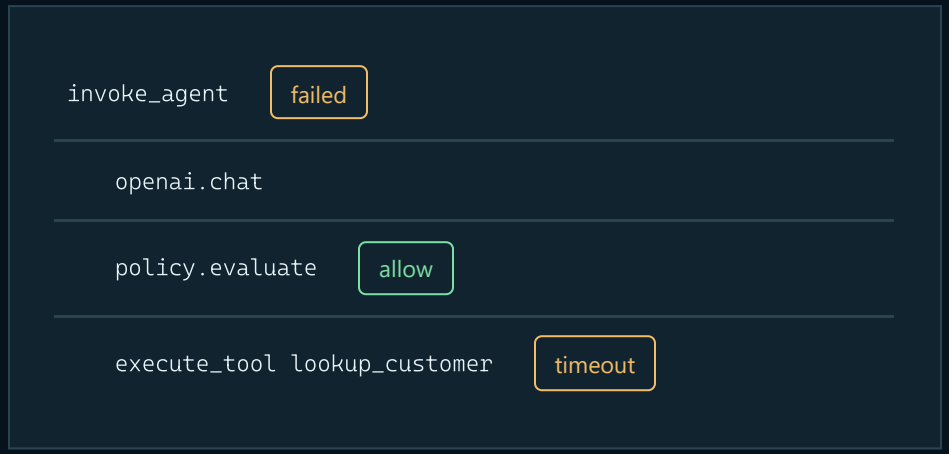
A missing span alone does not prove non-execution. Check the execution counter or resulting state too.

Policy denial is a business outcome, not automatically a runtime error.

Live demo · 3 Run lookup with timeout

Allowed does not mean successful

> Same lookup request. Inject a customer-service timeout.



Investigate the difference

Policy outcome: **allow**

Error type: **TimeoutError**

Time spent in the tool body

Runtime failures counted separately

Metrics reveal the pattern. Traces locate it. Logs explain it.

Controlled timeout simulation; no real downstream outage.

Make the evidence useful in production

Keep correlation intact

Preserve trace context across tools and services.
Connect policy logs to the relevant run.

Capture content deliberately

Model, duration and tokens are often enough.
Prompt and result capture requires deliberate handling.

Separate outcomes

Denied action ≠ failed execution.
Token usage ≠ cost or answer quality.

Verify the pipeline

Traces, logs and metrics each need export.
Sampled diagnostics are not complete audit evidence.

Aspire standalone is a development and diagnostic viewer. Production storage, retention and access controls need a deliberate backend design.

Control the action. Understand the run.

Enforce policy before execution.

Record the requested action, the decision and whether the tool body ran.

Connect traces, logs and metrics.

Use each signal to answer a different operational question.

Use shared conventions; verify actual output.

OpenTelemetry and GenAI conventions make the approach portable.

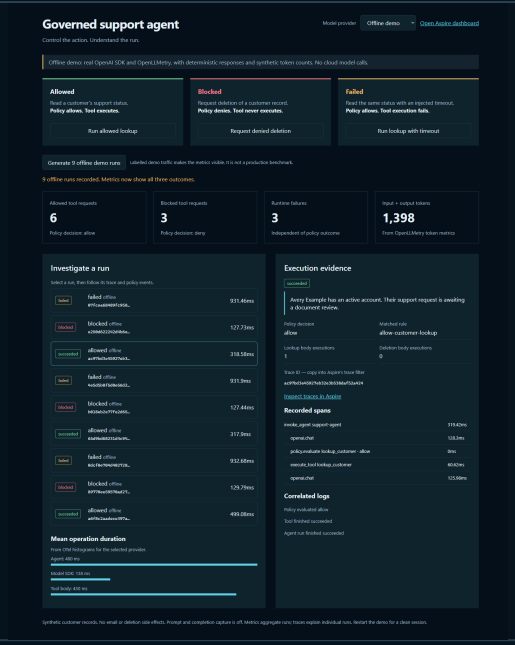
Demo · Dashboard · Sources and backup material follow.

Sources and implementation notes

- Microsoft Agent Governance Toolkit — runtime policy engine and host enforcement.
- OpenLLMetry — Python instrumentations; a separate JS/TS project is also available.
- OpenTelemetry GenAI semantic conventions — spans, metrics, events and provider conventions.
- OpenTelemetry Python — trace, metric and log pipelines.
- Aspire standalone dashboard — OTLP viewer for Python and other languages.

Pinned versions and source checks are in the repository. This demo uses Chat Completions to exercise both automatic model tracing and metrics.

Backup: the actual demonstration



Recorded offline rehearsal. All customer records and model usage counts are synthetic; the telemetry is collected through the real SDK and OpenLLMetry.

Use if a live tab or dashboard is unavailable.

Backup: approvals and standards

Human approval

A policy can require approval.
The host pauses and resumes execution.

Approval must be bound to the action and its arguments.

A resumed request may have a new trace. Preserve the relationship deliberately.

Conventions evolve

Inspect actual emitted fields and units.

Use `app.*` for business policy metadata.

Trace export does not automatically configure metric export.

The newer GenAI draft and a pinned instrumentation may use different metric names.